STRATEGIC ISSUES

- A testing strategy can only succeed if some key issues are addressed:
- **Quantifiable Requirements:** Define product requirements in measurable terms before testing begins. This includes not just error detection but also qualities like portability, maintainability, and usability.
- **Clear Testing Objectives:** State objectives in measurable terms (e.g., test coverage, defect density, cost of fixing bugs, mean-time-to-failure, test work-hours).
- **User Understanding:** Build user profiles and use cases to focus testing on real-world usage. This reduces unnecessary effort.
- **Rapid Cycle Testing:** Plan short, iterative test cycles that deliver small, usable increments. Feedback from these cycles helps control quality.

STRATEGIC ISSUES

- **Robust Software Design:** Software should be able to detect certain errors itself (antibugging techniques) and support automated and regression testing.
- **Technical Reviews Before Testing:** Reviews can catch errors early, reducing the amount of testing needed.
- **Review Test Strategy and Cases:** Reviews should also check the testing plan itself to uncover inconsistencies or omissions.
- **Continuous Improvement:** Collect metrics during testing and use them to refine the process over time, applying statistical control methods.

Test Strategies for Conventional (Traditional) Software

- Conventional or traditional software refers to older styles of software development, used before object-oriented programming, web apps, and mobile apps became popular.
- **Architecture Types:**
  - **Hierarchical Architecture:** Software is **organized into modules** in **a tree-like structure**. Each module has a specific function, and higher-level modules call lower-level ones to perform tasks.
  - **Call-and-Return Architecture:** Software is **built around functions or procedures**. Each function does a task and then returns control to the calling function. This makes the system modular and easier to manage.
- These architectures are **not object-oriented** and represent the classic way software was structured in earlier application domains.

Different Strategies of Testing Conventional Software

- 3 types of Conventional Software Testing Approaches

1. Big Bang Approach

- Testing is delayed until the **entire system** is fully developed.
- The whole system is tested at once, without prior testing of individual components.
- **Drawback:** Errors are harder to trace because they could originate from any component, making debugging complex and time-consuming.
- **Use case:** Rarely recommended, typically considered extreme due to high risk.

Different Strategies of Testing Conventional Software

2. Continuous Testing

- Testing occurs **daily or regularly** whenever new parts of the system are built.
- Ensures **early detection of errors**, reducing cost and effort in fixing them.
- Helps verify that components integrate smoothly as development progresses.
- **Benefit:** Saves time and resources in the long run by preventing error accumulation.

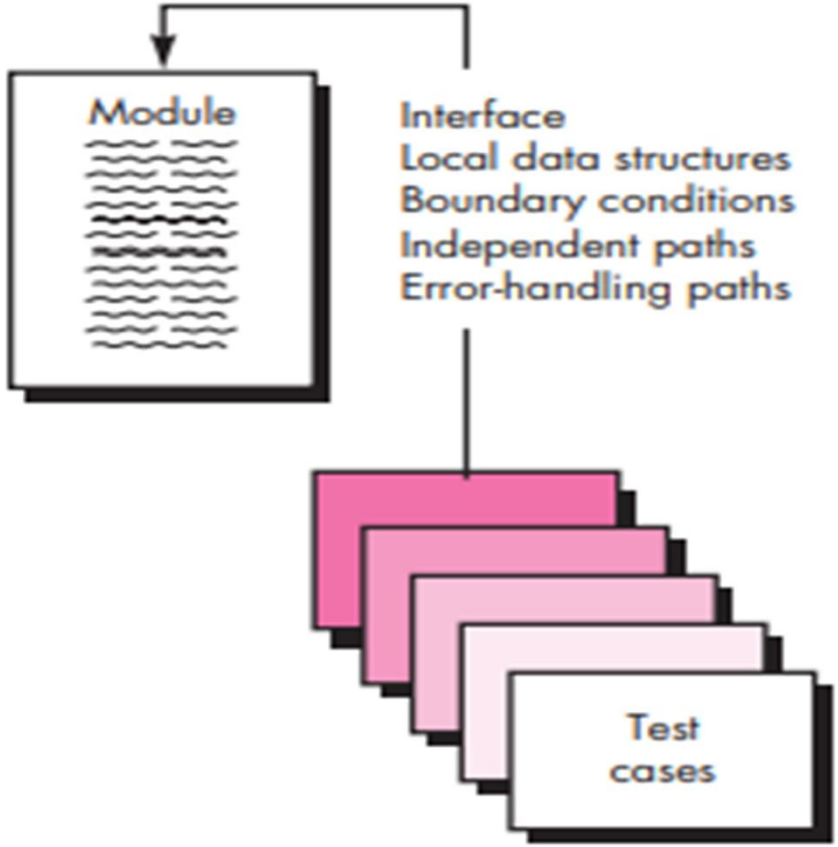
3. Incremental Testing

- Testing is performed in **stages**, starting small and gradually expanding.
- Process includes:
  - **Unit Testing:** Verifies individual program units work correctly in isolation.
  - **Integration Testing:** Combines units into subsystems and checks if they function together.
  - **System Testing:** Conducts end-to-end testing to ensure the entire system works as expected.
- **Benefit:** Errors are caught early at each stage, ensuring reliable integration and smoother final system testing.

Unit Testing

- **Definition:** Unit testing is a software testing method that verifies the smallest parts of a program, known as components or modules.
- **Purpose:** The goal is to **detect errors within a specific** module by testing critical control paths based on how the component is designed to function.
- **Scope:** It is limited in scope, **focusing only on one part** of the program at a time. The emphasis is on how the component processes information and manages data.
- **Efficiency**: **Multiple components can be tested simultaneously**, which helps save time and resources during development.

Unit Test Considerations



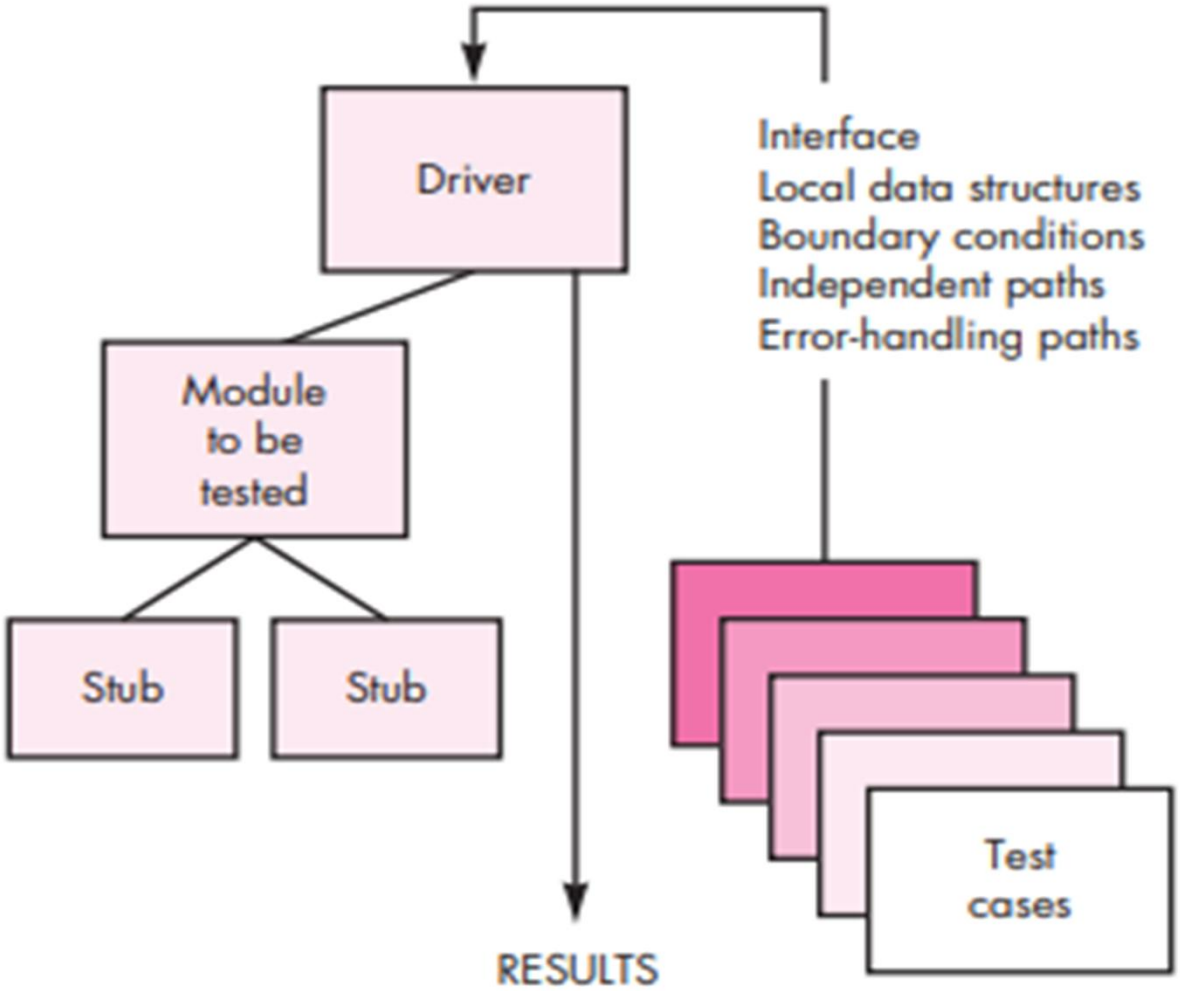
Unit Test Considerations

- By following these principles, developers can uncover errors early in the development cycle when fixes are easier and less costly ultimately improving software quality and reliability.
- **1. Module Interface**
- The **interface** defines how a module communicates with other program components.
- Inputs and outputs must be tested to confirm that data flows correctly into and out of the unit.
- Ensures the module processes information accurately and integrates smoothly with the rest of the system.
- **2. Local Data Structures**
- These are **temporary data structures** used during execution.
- Testing verifies that data stored temporarily maintains integrity throughout the algorithm's steps.
- Prevents issues such as data corruption, incorrect retrieval, or loss.

Unit Test Considerations

- **3. Independent Paths**
- Refers to all possible **control paths** within a module that can be executed independently.
- Each path should be tested to ensure every statement in the module is executed at least once.
- Provides thorough coverage and helps detect hidden logic errors.
- **4. Boundary Conditions**
- Boundaries are the **limits or restrictions** set for processing within a module.
- Testing ensures the module behaves correctly at these extremes (e.g., minimum/maximum values).
- Prevents unexpected behavior or failures when inputs approach edge cases.
- **5. Error-Handling Paths**
- Focuses on how the module responds to **errors and exceptions** during execution.
- All error-handling routines should be tested to confirm the module can recover gracefully.
- Ensures the program avoids crashes and prevents cascading failures in the system.

Unit Test Procedure



The steps involved in Unit-Test Procedures are

- Unit testing follows a systematic process to ensure that individual modules function correctly before integration. The key steps are:
- **1. Design of Unit Tests**
- Tests are designed to verify the functionality of the code.
- This can be done **before coding begins** (test-driven development) or after source code is generated.
- **Example:** For a function that calculates square roots, test cases might include positive, negative, and zero inputs to validate correctness across scenarios.
- **2. Test Cases and Expected Results**
- Each test case must be paired with a **set of expected results**.
- This ensures thorough testing and helps identify errors early.
- **Example:** Testing the square root function with input 4 should yield an expected output of 2.

The steps involved in Unit-Test Procedures are

- **3. Development of Drivers and Stubs**
- Since modules are not standalone programs, **drivers and stubs** are often required.
  - **Driver:** A simple main program that feeds test data to the module and displays results.
  - **Stub:** A placeholder module that simulates the behavior of subordinate components.
- **Example:** If the square root function relies on an input validation module, a stub can be created to simulate valid and invalid inputs.
- **4. Unit Test Environment**
- A controlled environment is set up to test modules in **isolation**.
- Drivers and stubs are used to mimic interactions with other components.
- This ensures the module works as expected before integration into larger systems.
- **5. Testing Process**
- Once the environment is ready, developers execute tests using various inputs and scenarios.
- **Example:** The square root function can be tested with inputs like 4, -4, and 0 to confirm correct handling of normal, invalid, and edge cases.

Integration Testing

- Even if all modules pass **unit testing**, the overall system may still fail when modules are combined.
- Issues can arise due to:
  - Data loss across interfaces
  - Unexpected side effects between components
  - Sub-functions not producing the intended overall functionality
- **Importance of Unit Testing**
- Thorough unit testing of **critical and complex components** helps detect problems early.
- This is especially valuable when development resources are limited.
- **What is Integration Testing?**
- **Integration testing** verifies the interactions between different modules or components.
- Its goal is to uncover problems that occur when modules work together.
- **Incremental Integration Testing**
- Components are integrated and tested **step by step**, gradually forming the complete system.
- This approach makes it easier to **identify and isolate issues**.
- **Subtypes of Incremental Integration Testing**
- **Top-Down Integration Testing**
  - Testing begins with higher-level modules and progressively integrates lower-level ones.
- **Bottom-Up Integration Testing**
  - Testing starts with lower-level modules and builds upward toward the complete system.

1.Top-Down Integration Testing

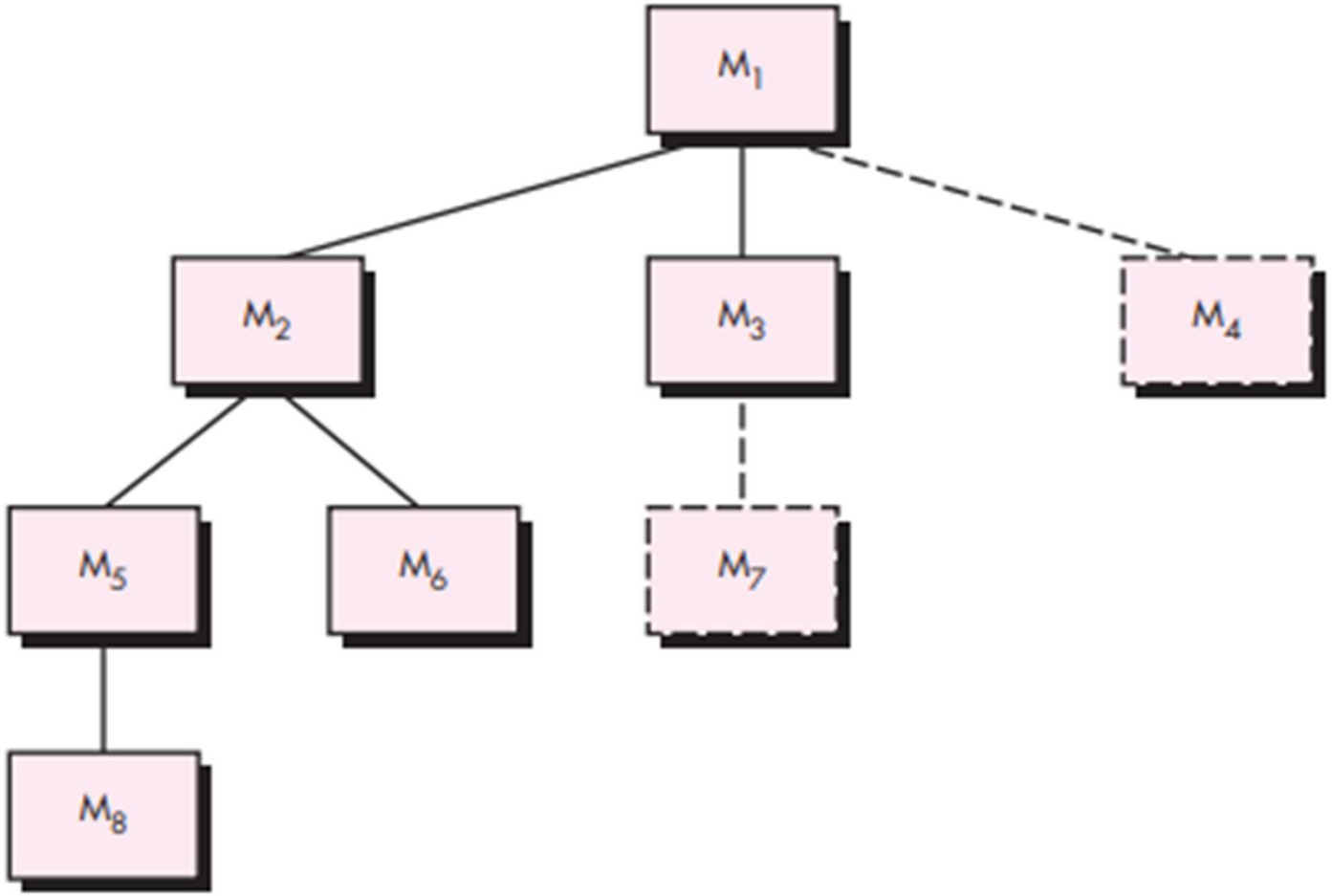
- **Top-down integration testing** is an incremental approach to building and testing software architecture.
- Modules are integrated by moving **downward through the control hierarchy**, starting with the **main control module**.
- Subordinate modules are added in either a **depth-first** or **breadth-first** manner.

Steps in the Process

- **Main Control Module as Test Driver**
  - The main control module is used as the driver.
  - Stubs replace all directly subordinate components.
- **Stub Replacement**
  - Depending on the chosen approach (depth-first or breadth-first), stubs are replaced one at a time with actual components.
- **Testing During Integration**
  - Each component is tested immediately after integration.
- **Progressive Replacement**
  - After completing tests for one component, another stub is replaced with the real module.
- **Regression Testing**
  - Regression tests may be performed to ensure that new errors are not introduced during integration
- The process continues until the **entire program structure** is built.
- This strategy emphasizes verifying **major control and decision points early**.
- Detecting control-related problems at the start helps prevent costly issues later in development.

.

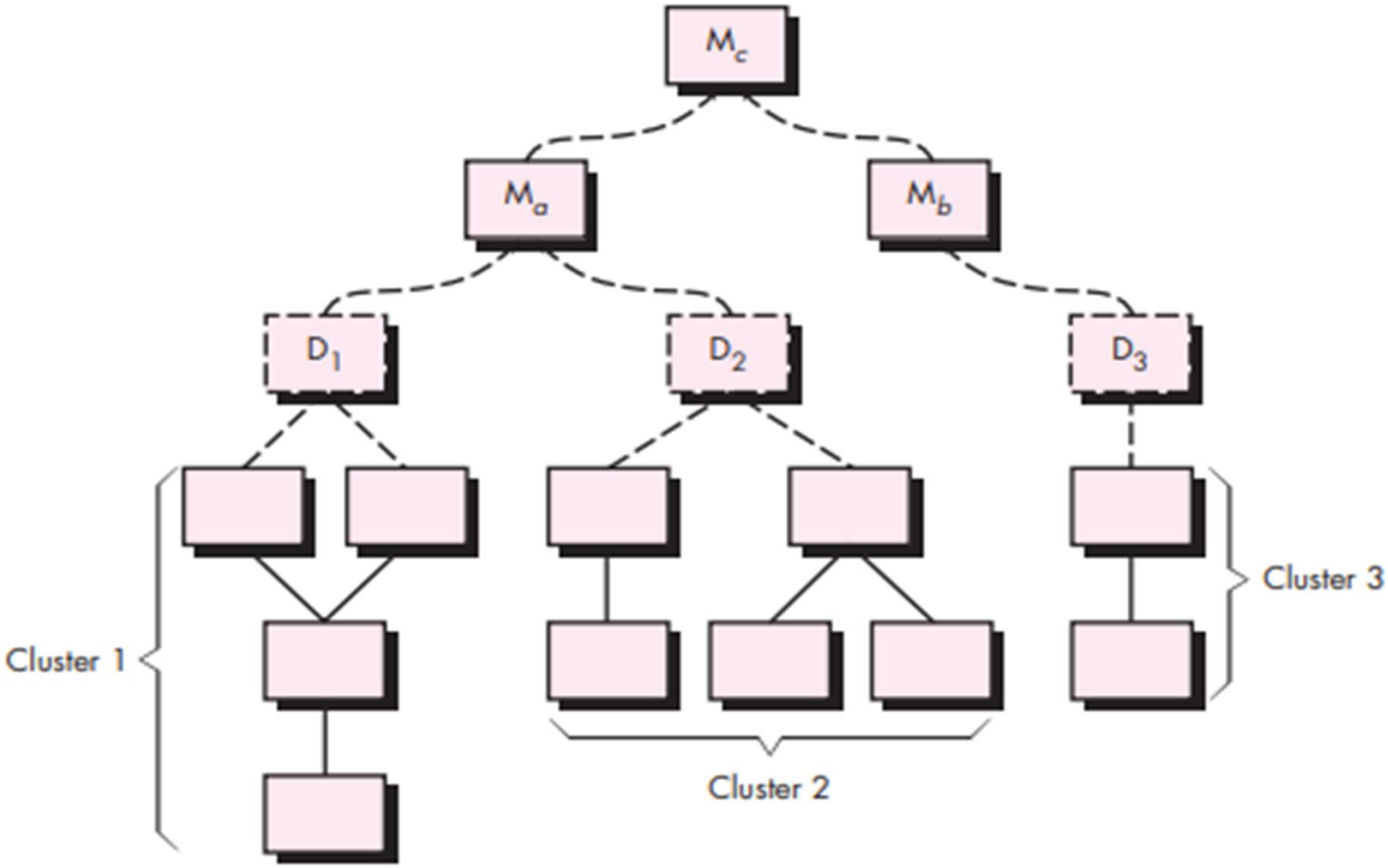
1.Top-Down Integration Testing



2. Bottom-Up Integration Testing

- **Bottom-up integration testing** begins with the **smallest or atomic components** of the program.
- These components are tested individually, then combined into **clusters** that perform specific sub-functions.
- A **driver program** is used to supply test inputs and capture outputs during cluster testing.
- As clusters are validated, drivers are gradually removed, and clusters are integrated upward into larger structures until the full system is built.
- **Steps in the Process**
- **Form Clusters**
  - Combine small components into clusters that perform specific functions.
- **Create Driver Programs**
  - Write drivers to coordinate test-case input and output for each cluster.
- **Test Clusters**
  - Execute tests to ensure each cluster works correctly.
- **Integrate Upward**
  - Remove drivers and progressively combine clusters into larger ones, moving upward in the program hierarchy.
- Focuses on validating **low-level functionality first**.
- Ensures that foundational components are reliable before building higher-level structures.
- Particularly effective when lower-level modules are complex or critical to system stability.

2. Bottom-Up Integration Testing



Regression Testing

Definition

- Regression testing is the **re-execution of a subset of test cases** (or automated capture/playback tests) to confirm that recent changes haven't caused side effects.
- It is a safeguard against defect reintroduction during ongoing development and integration.

Why It's Needed

- Each time a new module is added during integration testing, the software changes.
- These changes can unintentionally break functions that previously worked correctly.
- **Regression testing** ensures that modifications do not introduce new defects or cause old ones to reappear.

Regression Testing

Classes of Regression Test Cases

- **Representative Sample**
  - Tests that exercise all major software functions.
- **Change-Sensitive Tests**
  - Additional tests focusing on functions most likely to be affected by recent changes.
- **Directly Modified Components**
  - Tests targeting the specific modules or components that were altered.

Benefits of Regression Testing

- **Early Problem Detection** – catches issues before they spread.
- **Risk Reduction** – minimizes the chance of introducing new defects.
- **Cost & Time Savings** – prevents expensive fixes later in development.
- **Improved Quality** – ensures the system remains stable and reliable.
- **Confidence Boost** – increases trust in the software's overall reliability.

Smoke Testing

- **Smoke testing** is a quick check performed in software development to ensure that a build works as intended.
- The term comes from the idea of “lighting a match” to see if smoke appears—similarly, smoke testing quickly reveals whether a build has **major issues** that would prevent it from functioning.
- **Process**
- Smoke testing typically involves three main activities:
- **Integrating Components into a Build**
  - All software components (files, libraries, reusable modules, engineered parts) are combined into a build.
  - The build represents a working version of the product with one or more functions implemented.
- **Designing and Running Tests**
  - A set of tests is created to uncover critical errors (“show-stoppers”) that could block progress.
  - The focus is on detecting issues that would prevent the build from working properly.
- **Daily Smoke Testing**
  - Builds are integrated and tested **every day**.
  - This ensures that problems are caught early and progress can be tracked continuously.

Smoke Testing

Definition

- Smoke testing is a **software testing technique** used to quickly verify whether a build has any major defects.
- It involves integrating components, running a basic set of tests, and repeating the process regularly to maintain stability.

Benefits of Smoke Testing

- **Minimizes Integration Risk** – reduces the chance of major failures when modules are combined.
- **Improves Product Quality** – ensures stability from the earliest stages.
- **Simplifies Error Diagnosis** – makes it easier to identify and fix problems quickly.
- **Tracks Development Progress** – provides a daily measure of how well the system is functioning.

System Testing

- **System testing** is a critical phase performed after **integration testing**.
- It involves testing the **entire system as a whole** to ensure it meets all specified requirements and functions correctly in its intended environment.
- **Objectives**
- Verify that the software satisfies both **functional** and **non-functional requirements**, including:
  - Performance
  - Reliability
  - Usability
  - Security
  - Compatibility
- Ensure the system behaves as expected under real-world conditions.

System Testing

Activities in System Testing

- **Test Planning**
  - Define the scope of testing.
  - Identify test cases and create a detailed test plan.
- **Test Case Development**
  - Design comprehensive test cases covering all aspects of system functionality.
- **Test Execution**
  - Run the test cases and record results.
- **Defect Management**
  - Track defects found during testing.
  - Prioritize issues based on severity.
  - Collaborate with developers to resolve them.
- **Reporting**
  - Summarize testing outcomes.
  - Communicate results to stakeholders for informed decision-making.

Importance

- Confirms that the software meets **all requirements** and works correctly in its intended environment.
- Identifies defects early, reducing risks before release.
- Enhances confidence in the system's overall quality and readiness for deployment.

Test Strategies For Object-Oriented Software

- The goal of testing is to find as many errors as possible within a reasonable amount of time and effort. .
- This basic purpose does not change for object-oriented software. However, the unique features of object-oriented systems require different testing strategies and methods.

Unit Testing in the OO Context

- In object-oriented programming, the idea of a unit testing is different
- **Unit definition changes**
  - In object-oriented programming, a "unit" is not just a single function or procedure.
  - Classes and objects, defined by encapsulation, become the primary units of testing.
- **Encapsulation impact**
  - Each class contains both data and the methods that manipulate that data.
  - Testing must consider the interaction between data and operations within the class.
- **Focus of unit testing**
  - Unit testing usually targets classes rather than isolated functions.
  - The smallest testable parts inside a class are its methods.
- **Shift from conventional view**
  - Traditional unit testing tests a single operation in isolation.
  - In object-oriented software, this approach is less effective.
  - Operations should be tested as part of the class context, ensuring they work correctly with the class’s data and other methods.

Integration Testing in the OO Context

- Integration testing in object-oriented (OO) software presents **unique challenges** because of the absence of a clear hierarchical control structure and the complex interactions among components.
- Traditional strategies like **top-down** or **bottom-up** integration often fall short, since operations cannot always be integrated one at a time due to direct and indirect

Integration Testing in the OO Context

- To address these challenges, other approaches such as
- **1.Thread-based testing**: Simulates different execution threads to uncover issues related to concurrency and synchronization.
- **2.Use-case-based testing**: Designs test cases around real-world scenarios and system use cases, ensuring that interactions reflect actual usage patterns.

Integration Testing in the OO Context

- **3.Cluster Testing**
- Cluster testing is central to OO integration. It focuses on groups of collaborating classes, identified through **CRC (Class-Responsibility-Collaboration)** cards and object-relationship models. Test cases are designed to expose errors in these collaborations.

Supporting Techniques

- **Drivers**: Used to test operations at the lowest level or entire groups of classes.
- **Stubs**: Employed when collaboration between classes is required but some classes are not yet fully implemented.

Validation Testing

- Validation testing starts **after integration testing** is finished.
- By then, all components are tested, the software is fully assembled, and interface errors are fixed.
- At the system level, differences between conventional, object-oriented, and web applications don't matter.
- Testing focuses on **user-visible actions** and **user-recognizable outputs**.
- Validation means checking if the software behaves as the customer would reasonably expect.
- The **Software Requirements Specification (SRS)** is the reference point, since it lists user-visible features and includes validation criteria.
- **Validation testing should be carried out through three steps:**

1)Validation Test Criteria

2)Configuration Review

3)Alpha And Beta Testing

1 Validation-Test Criteria

- Validation ensures the software meets all requirements.
- A **test plan** defines classes of tests; a **test procedure** defines specific test cases.
- Tests check:
  - Functional requirements
  - Behavioral characteristics
  - Content accuracy and presentation
  - Performance requirements
  - Documentation correctness
  - Usability and other qualities (transportability, compatibility, error recovery, maintainability)
- After each test:
  - If results match specifications → accepted.
  - If deviations occur → deficiency list created.
- Errors found here are hard to fix before delivery, so negotiation with the customer may be needed.

2 Configuration Review

- A **configuration review (audit)** ensures:
  - All software elements are properly developed.
  - Components are cataloged.
  - Details are sufficient to support ongoing activities.
- Acts as a formal check before release.

3 Alpha and Beta Testing

- Developers cannot fully predict how customers will use software.
- **Acceptance testing** (for custom software):
  - Conducted by end users, not engineers.
  - Can be informal (“test drive”) or formal, lasting weeks/months.
  - Helps uncover cumulative errors.
- **Alpha testing**:
  - Done at developer’s site.
  - End users test in a controlled environment.
  - Developer observes and records issues.
- **Beta testing**:
  - Done at customer sites.
  - Developer not present.
  - Customers report problems regularly.
  - Issues are fixed before final release.
- **Customer acceptance testing** (variation):
  - Formal tests by the customer before accepting software.
  - Often used in large corporate or government projects.
  - Can last days or weeks.

System Testing

- System testing is a crucial phase of software testing that follows integration testing. It evaluates the entire system as a whole to ensure it meets specified requirements and functions correctly in its intended environment.
- Unlike unit or integration testing, system testing involves not just software engineers but also considers hardware, people, and information working together as a complete system.
- The primary objective of system testing is to verify that the system satisfies both **functional** and **non-functional requirements** such as performance, reliability, usability, security, and compatibility.

Types of System Testing

1. Recovery Testing

- Ensures the system can recover from unexpected failures or errors.
- Failures are deliberately introduced to check if the system restores functionality without losing critical data.
- Important for maintaining reliability in unpredictable scenarios.

2. Security Testing

- Evaluates the system's protection against potential attacks.
- Focus areas: authentication, access control, encryption, and data protection.
- Identifies vulnerabilities so developers can patch them or add safeguards like firewalls and intrusion detection systems.

3. Stress Testing

- Determines how the system behaves under extreme or abnormal conditions.
- Measures tolerance limits before failure occurs.
- Monitors response times, errors, crashes, memory usage, and network traffic under heavy load.

Types of System Testing

- **4. Performance Testing**
- Assesses system efficiency under normal operating conditions.
- Key metrics: response time, throughput, CPU/memory usage, and network utilization.
- Helps identify bottlenecks and optimize code or resources to meet performance standards.
- **5. Deployment (Configuration) Testing**
- Validates system behavior across different environments and operating systems.
- Checks compatibility with hardware and other software components.
- Often conducted using virtual machines or cloud-based simulations.
- Ensures smooth installation, configuration, and operation in diverse deployment scenarios.

The Art of Debugging

- **Testing**: This is a **planned process** where we design test cases, follow a strategy, and check results against expectations. Testing shows us if something is wrong.
- **Debugging**: This happens after testing finds an error. Debugging is the **process of finding and fixing that error.**

Debugging is Called an Art

- Even though debugging can **follow a logical process**, it often requires **creativity, intuition, and experience**.
- You usually see **symptoms** (like a crash or wrong output), but the **cause** (like a wrong variable or logic error) may be hidden somewhere else in the code.
- Connecting the symptom to the actual cause is not always straightforward—it's a mental process that feels more like problem-solving art than strict science.

Debugging Process

- **Debugging vs. Testing**: Testing checks if the program works as expected. Debugging starts when testing shows a mismatch **between expected and actual results**.
- **Steps**:
  - Run a test case.
  - Notice that the output doesn't match what was expected.
  - This mismatch (symptom) points to a hidden problem (cause).
  - Debugging tries to connect the symptom to the cause and fix the error.
- **Outcomes**
- **Cause found and fixed** → The error is corrected.
- **Cause not found** → The developer makes guesses, designs new tests, and repeats until the cause is discovered.

Debugging Process

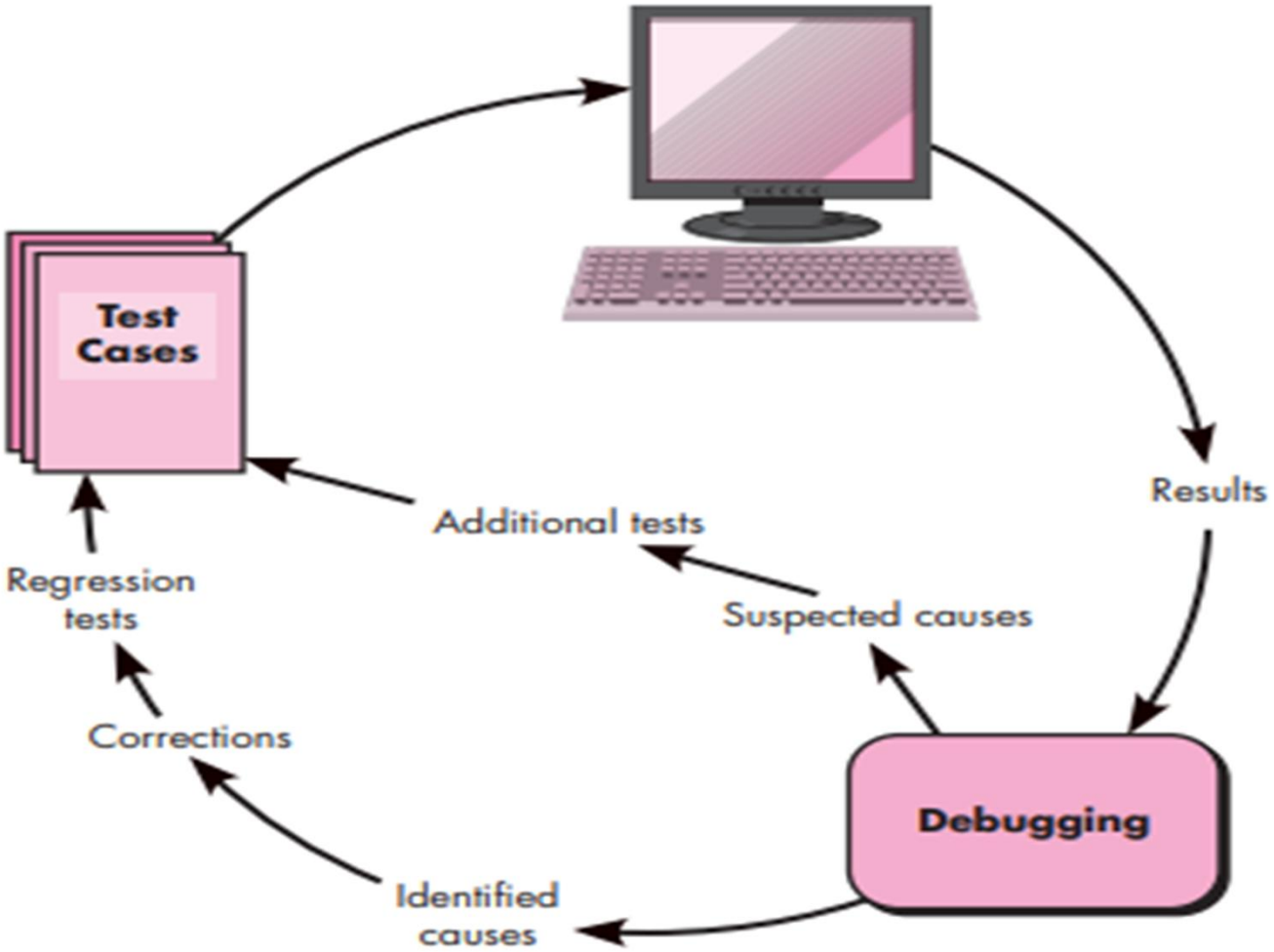
Debugging is Hard

- The **symptom and cause may be far apart** in the program.
- Fixing one error may **hide another temporarily**.
- Sometimes the issue is not a real bug (like rounding errors).
- Errors may come from **human mistakes** that are hard to trace.
- Problems may be due to **timing issues** instead of logic.
- Input conditions may be **hard to reproduce** (like in real-time systems).
- Some bugs are **intermittent** (appear only sometimes).
- Causes may be **spread across multiple tasks or processors**.

Impact of Errors

- Errors can range from small annoyances (like wrong formatting) to serious failures (like system crashes causing damage).
- The more serious the error, the greater the pressure to fix it quickly.
- Under pressure, developers may fix one bug but accidentally introduce new ones.

Debugging Process



Psychological Considerations in Debugging

- **Innate ability**: Some people naturally excel at debugging, while others struggle—even if they have the same training and experience.
- **Frustration factor**: Debugging can feel like solving puzzles, but with the added annoyance of knowing you made a mistake.
- **Emotional impact**: Anxiety, stress, and reluctance to admit errors can make debugging harder.
- **Relief**: Once the bug is fixed, tension drops and there’s a sense of satisfaction.

Automated Debugging Tools

- Modern tools (like IDEs, tracers, test-case generators) help catch errors faster.
- Examples: compilers that flag missing symbols, debuggers with graphical interfaces, bug trackers.
- Tools are helpful but **cannot replace clear design and clean code**.

The People Factor

- Sometimes the best debugging tool is **another person's perspective**.
- Pair programming or simply asking for help can reveal what you missed.

Correcting the Error

- Before fixing a bug, ask three key questions:
- **Is this mistake repeated elsewhere?** → Fix the pattern, not just one spot.
- **Will my fix cause new bugs?** → Be careful with highly connected code.
- **Could this bug have been prevented?** → Improve the process to avoid similar issues in the future.

RMMM Plan (Risk Mitigation, Monitoring, and Management)

- A **Risk Management Plan** in software projects helps identify, track, and handle risks. It has three parts: **Mitigation, Monitoring, and Management**.

1. **Risk Mitigation:** Prevent risks before they happen.

- **Steps:**
- Identify possible risks.
- Remove or reduce causes of risks.
- Keep documents updated.
- Review progress regularly.

RMMM Plan

2.Risk Monitoring

- Goal: **Track risks during the project.**
- Objectives:
- Check if predicted risks occur.
- Ensure mitigation steps are applied.
- Collect data for future analysis.
- Link problems to specific risks.

3.Risk Management

- Goal: **Handle risks when they actually occur.**
- Assumes mitigation failed and risk is real.
- Project manager responds with a plan (risk register).
- Adjust resources, schedules, and responsibilities to minimize damage.

RMMM Plan

- Example: Server Crash Risk in a Software Project.

Risk Mitigation

- Use reliable hardware and cloud services.
- Set up regular backups of project data.
- Test servers before project launch.
- Train staff to handle basic server issues.

Risk Monitoring

- Track server performance (CPU, memory, uptime).
- Monitor error logs and downtime reports.
- Check backup status regularly.
- Watch for signs of overload (too many users at once).

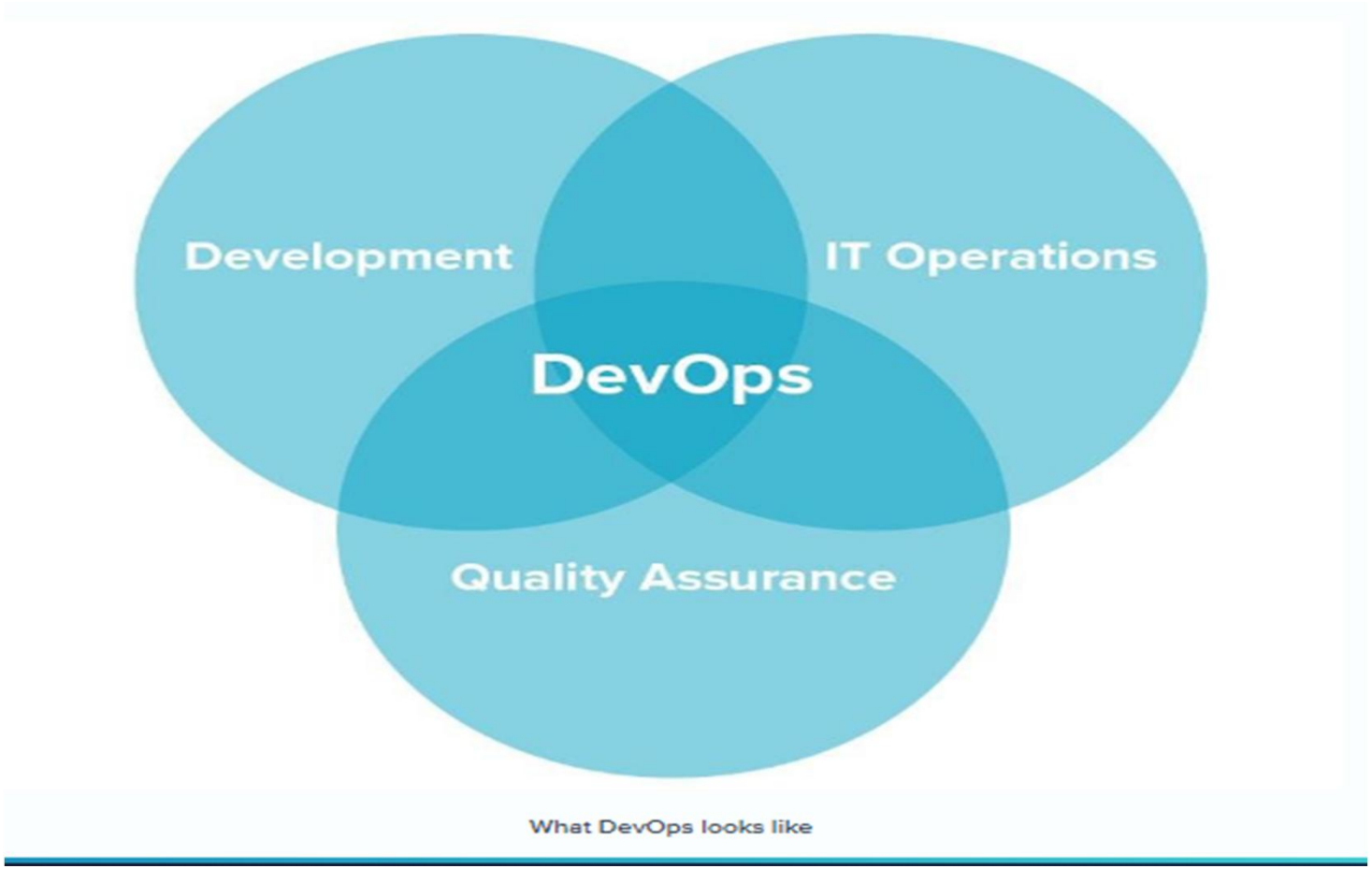
Risk Management

- If the server actually crashes:
  - Switch to backup servers immediately.
  - Restore data from backups.
  - Inform the team and adjust timelines.
  - Investigate the cause and prevent future crashes.

Introduction to DevOps

- DevOps is a software development practice that promotes collaboration between **development and operations**, resulting in faster and more reliable software delivery.
- Commonly referred to as a culture, DevOps connects people, process, and technology to deliver continuous value.

Introduction to DevOps



What DevOps looks like